

SimAI-OXC 集成设计文档

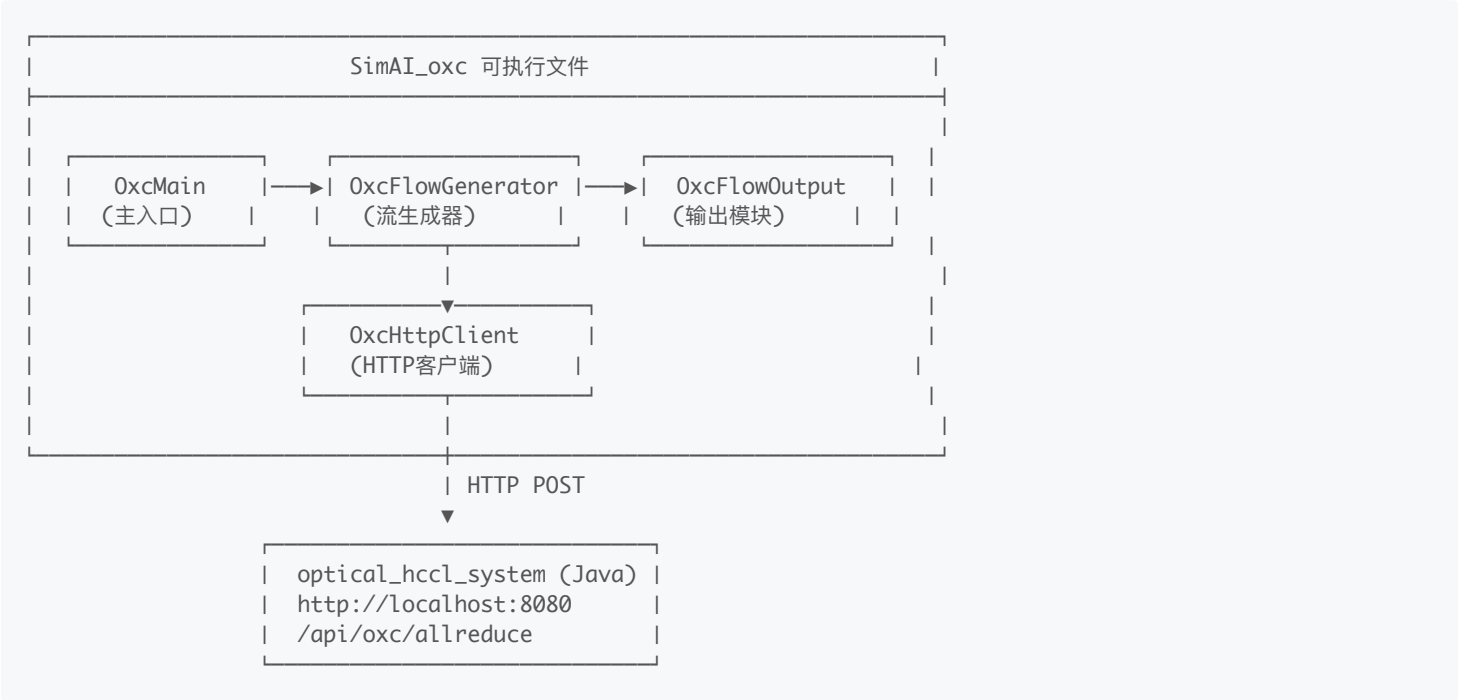
1. 概述

SimAI-OXC 是 SimAI 模拟器的扩展模块，用于集成 optical_hccl_system 的 OXC（光交叉连接）集合通信算法。该模块解析训练工作负载，调用 OXC HTTP API 生成 AllReduce 通信流，并输出流矩阵和依赖关系图。

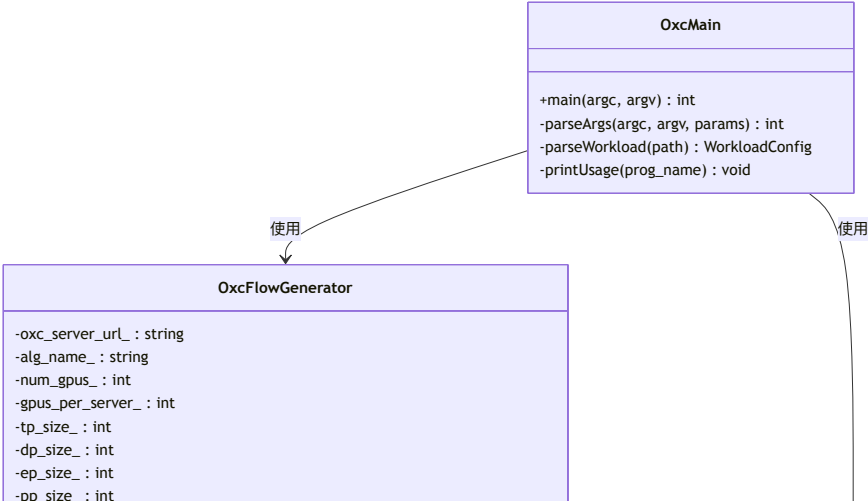
1.1 设计目标

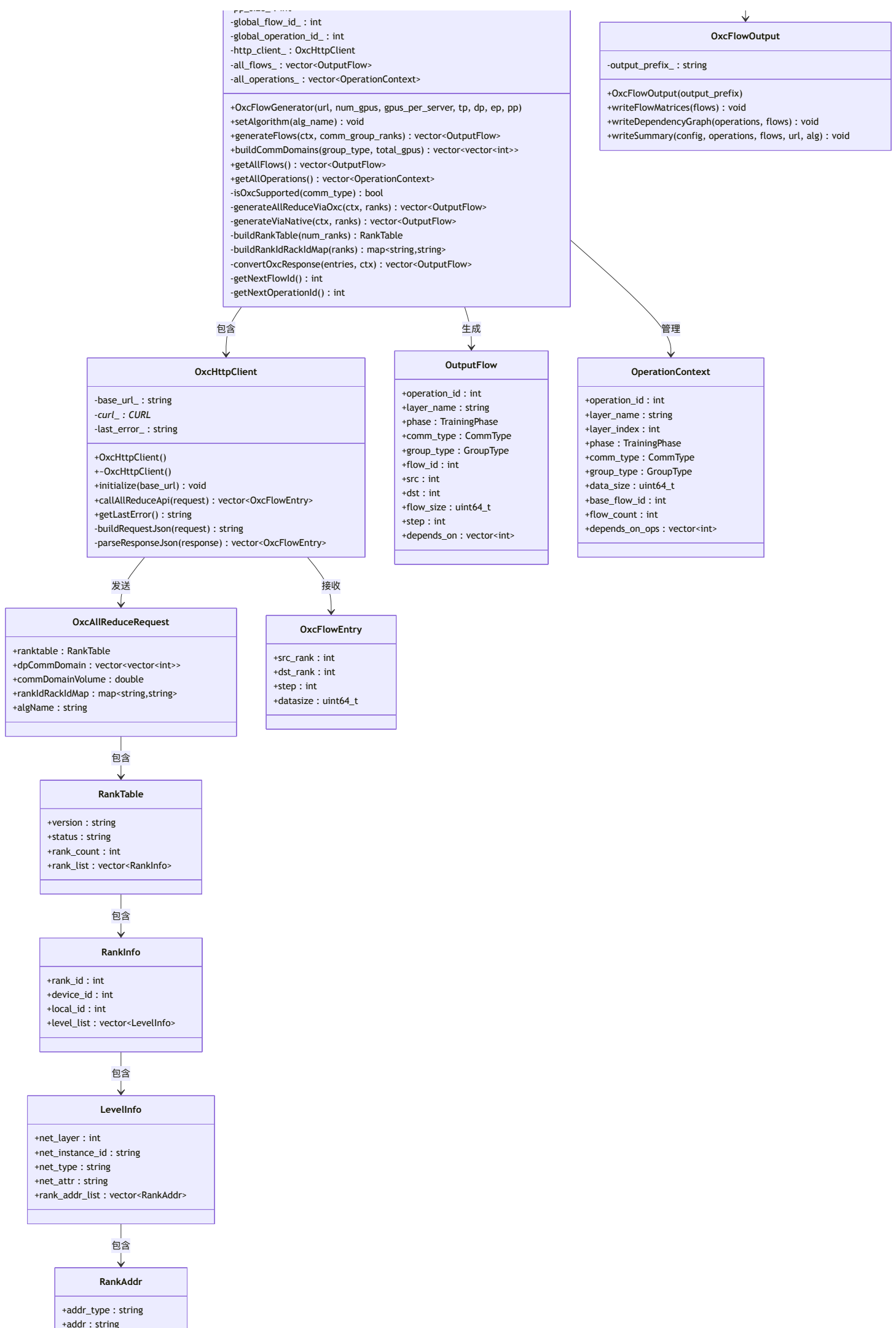
- 解析 SimAI 训练工作负载文件
- 对 AllReduce 操作调用 OXC API 获取优化的通信流
- 对其他集合通信操作使用原生 Ring 算法
- 生成流矩阵（CSV）和依赖关系图（JSON）
- 提供 `-c oxc` 编译选项

1.2 系统架构



2. 类图

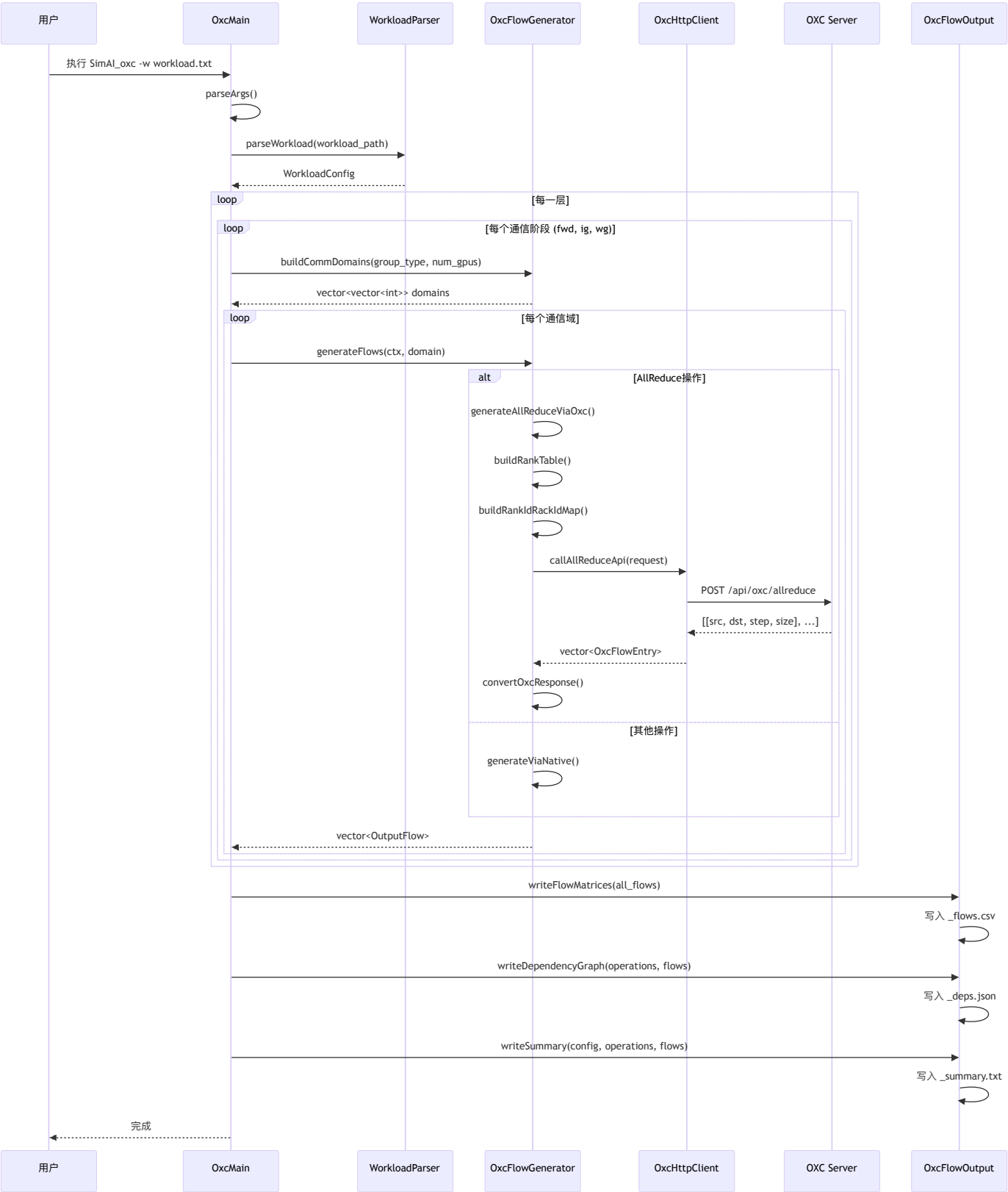




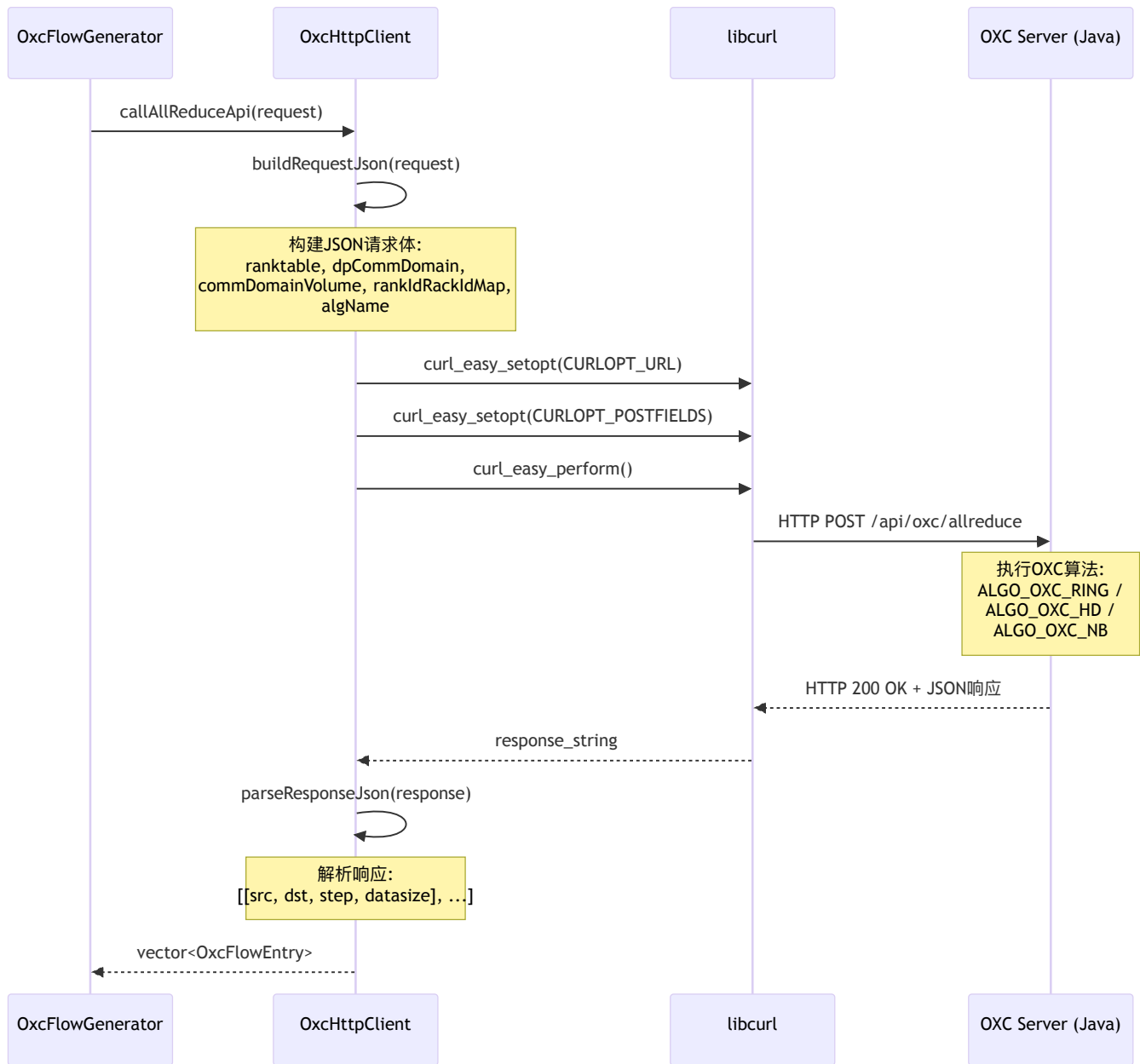
+ports : vector<string>
+plane_id : string

3. 时序图

3.1 主流程时序图

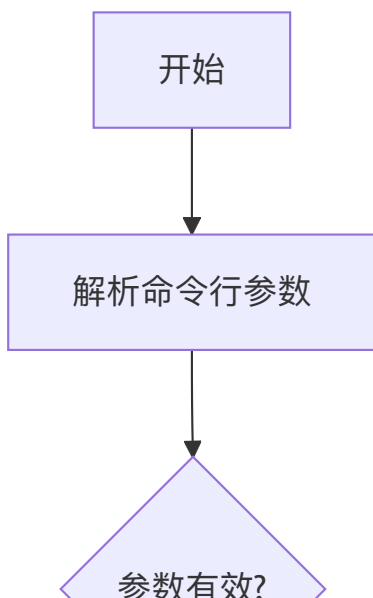


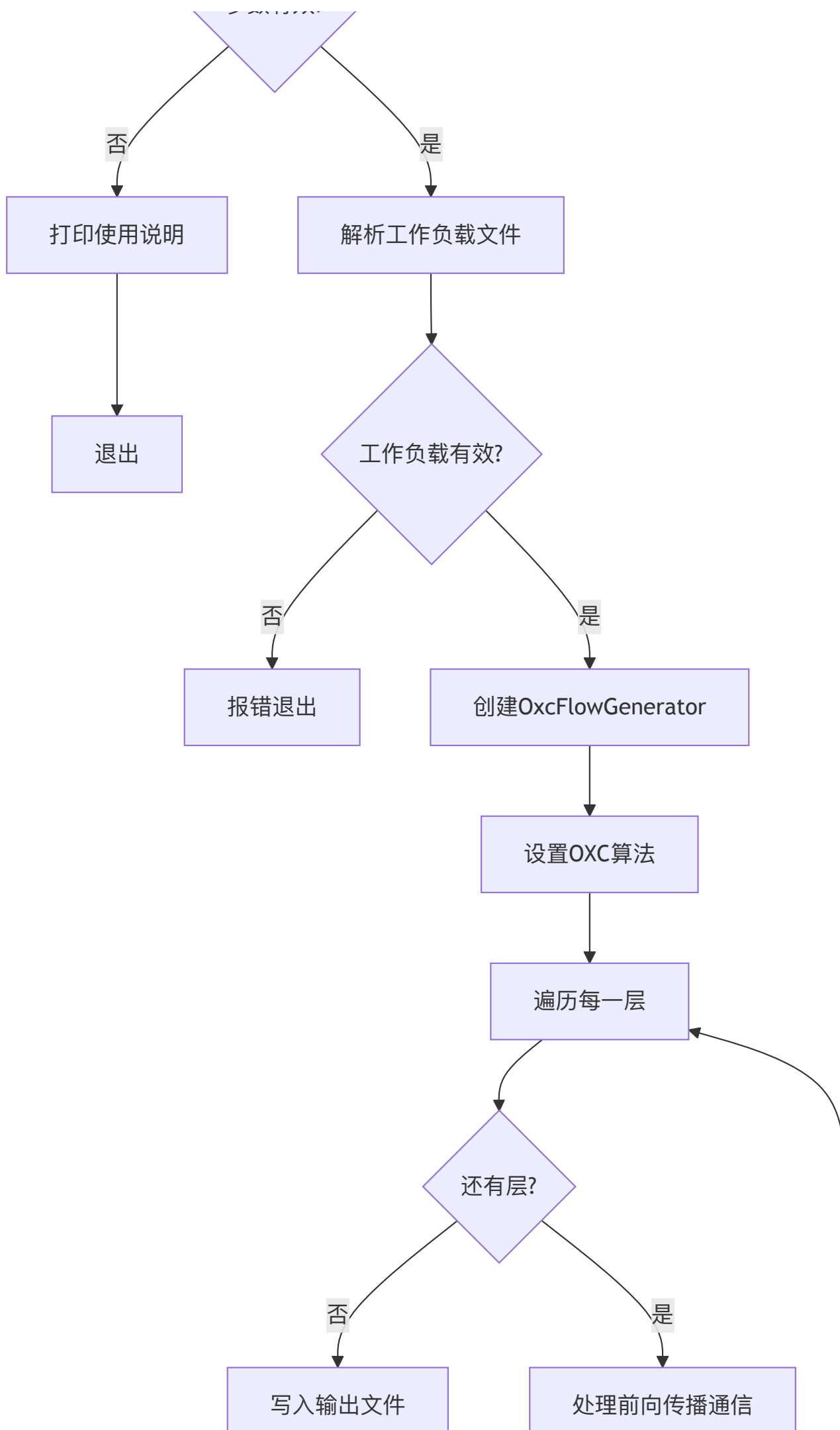
3.2 OXC API 调用时序图

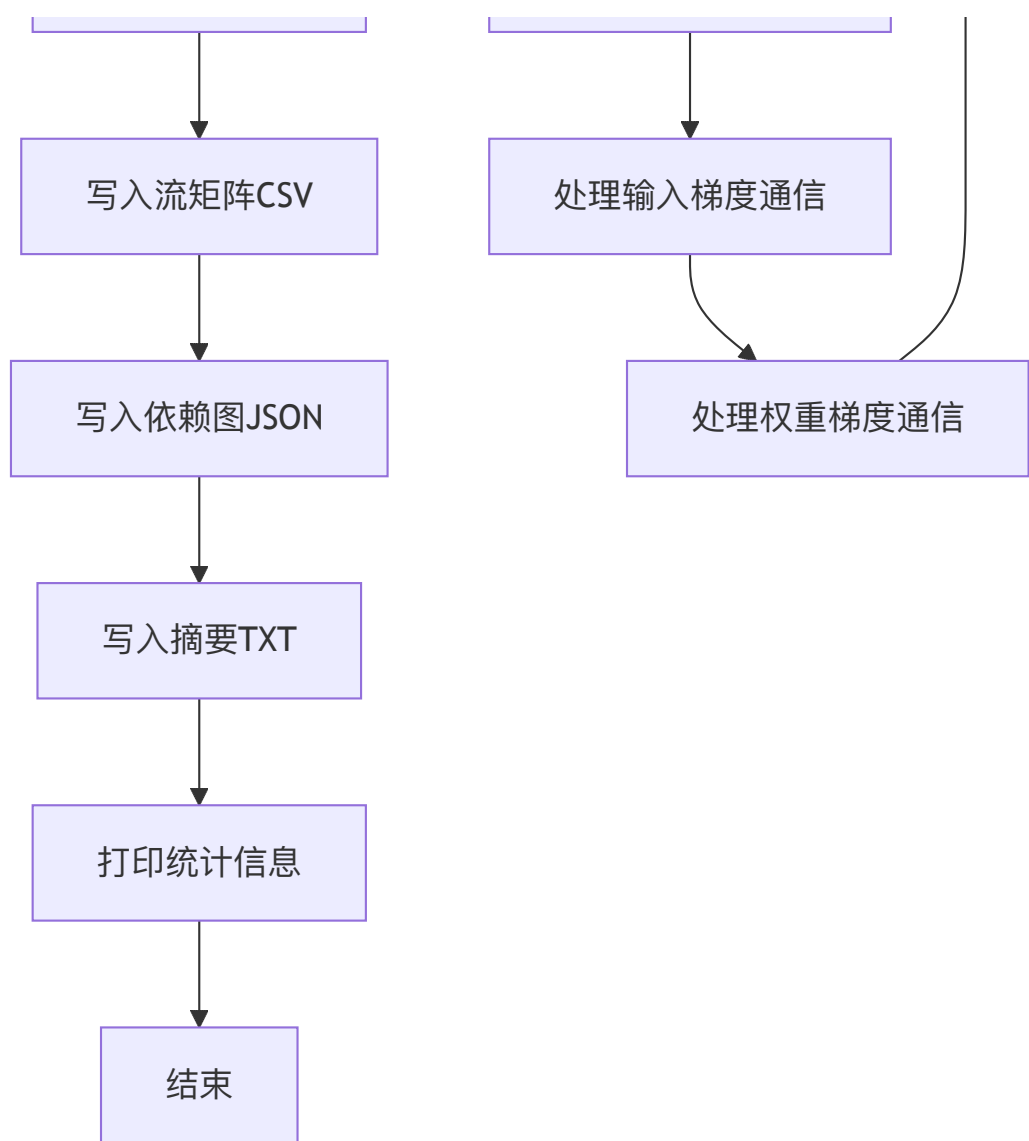


4. 流程图

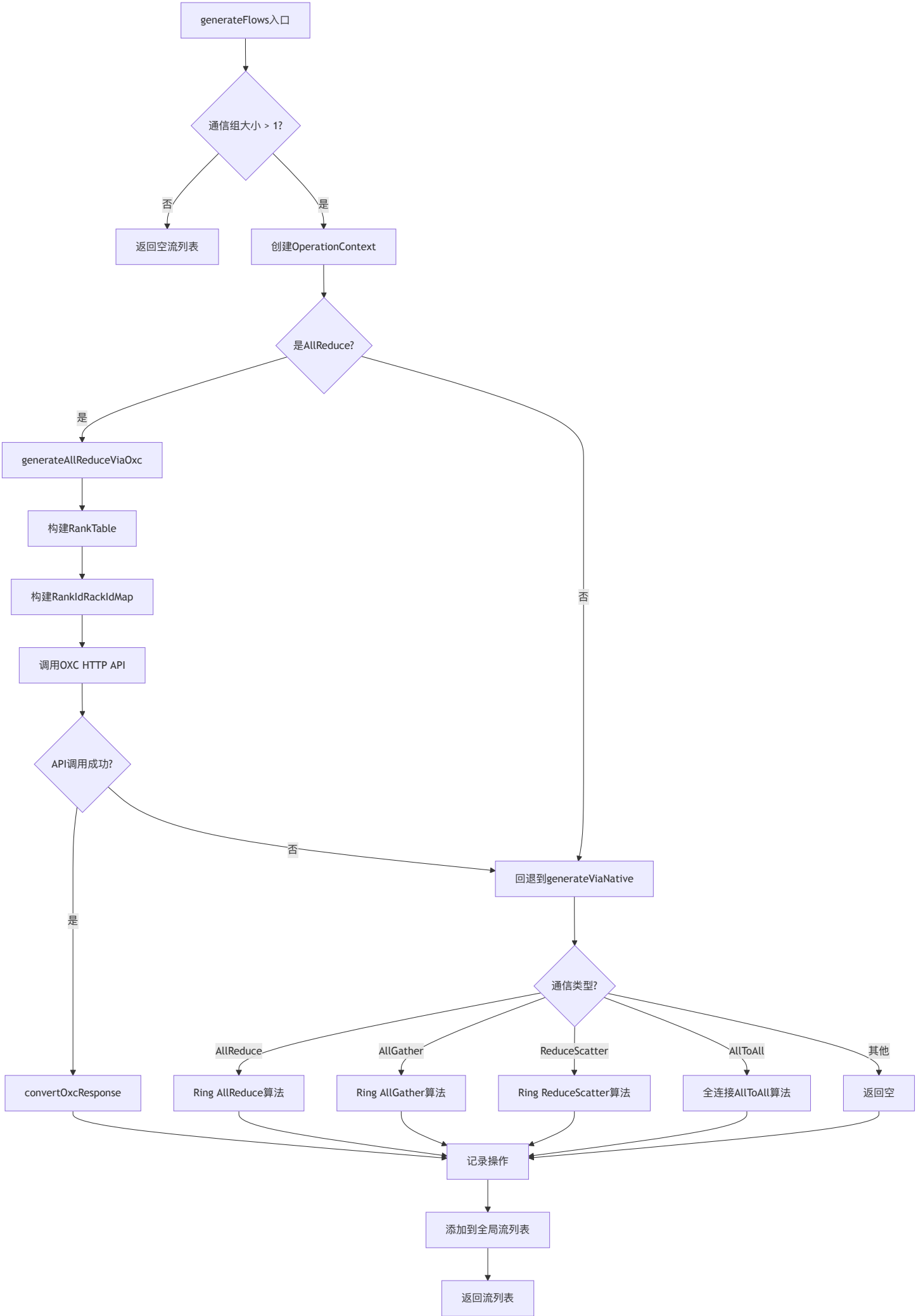
4.1 主程序流程图



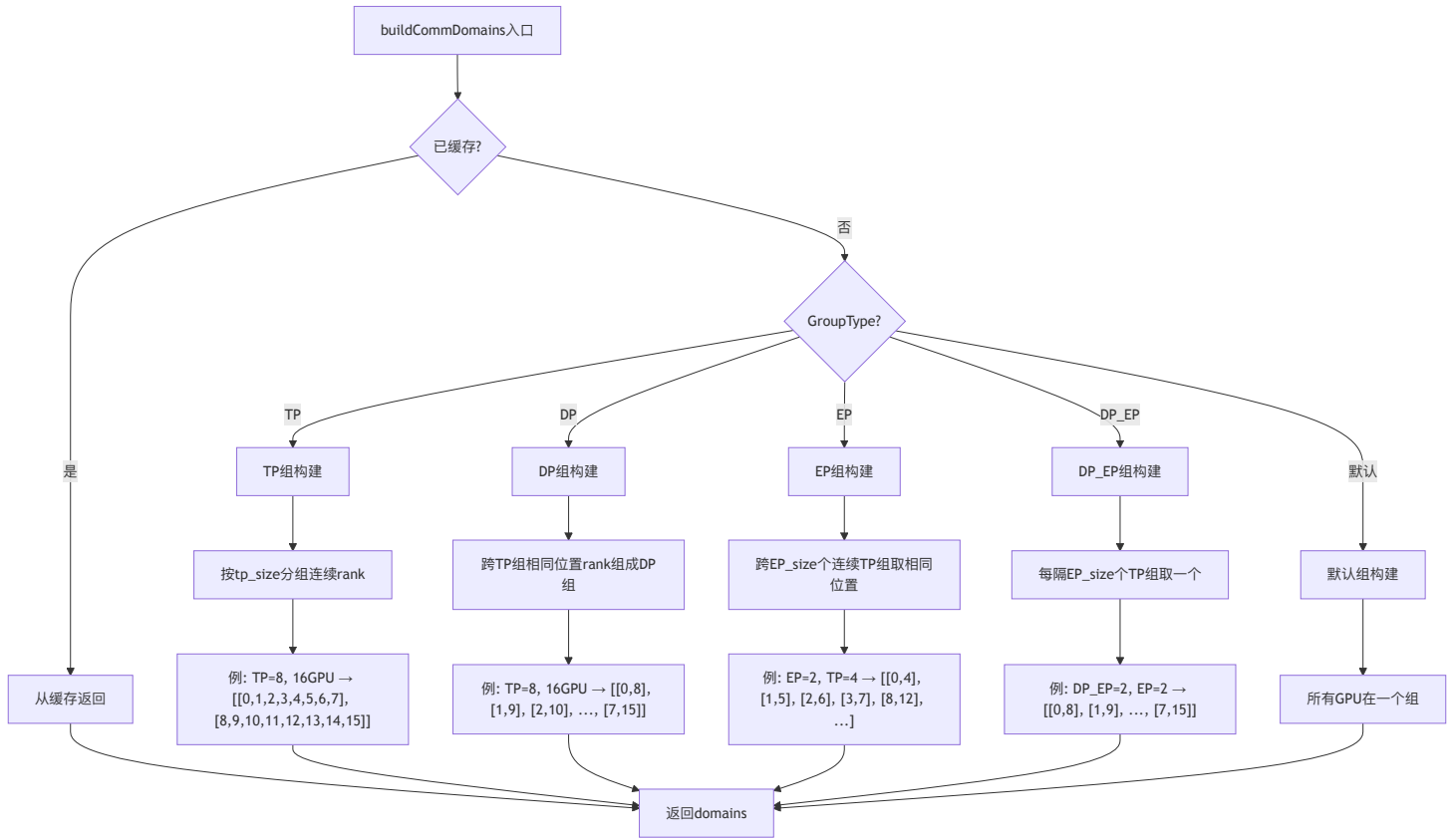




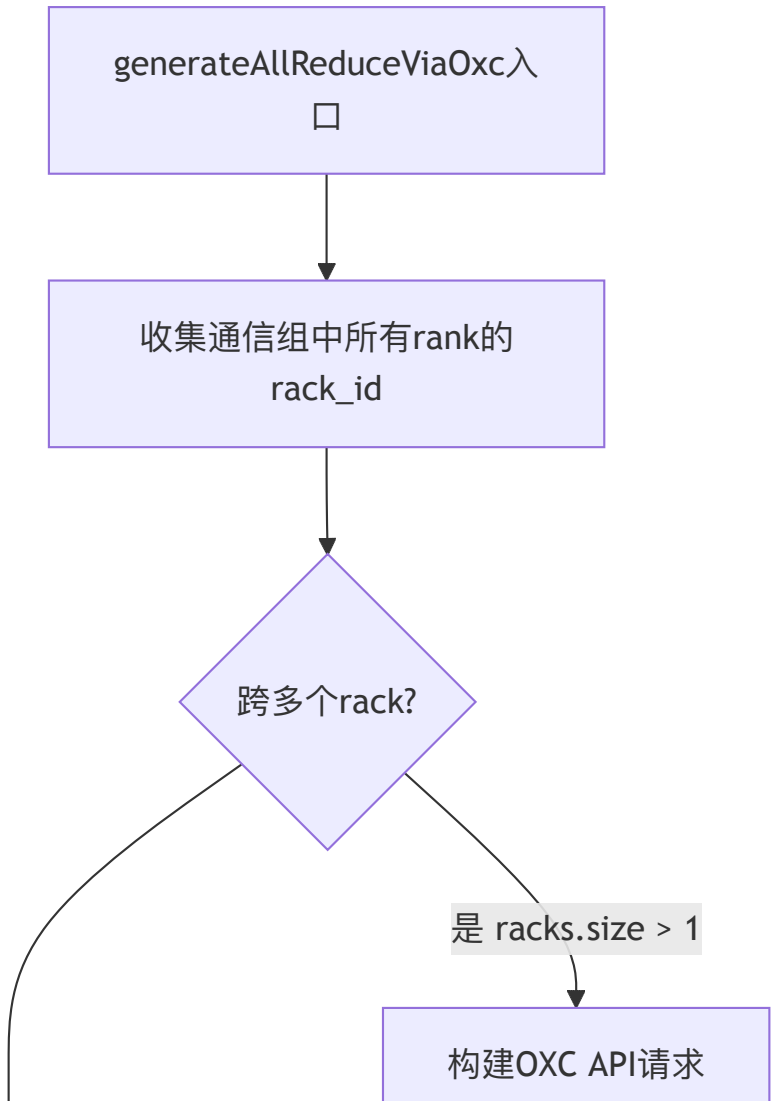
4.2 流生成决策流程图

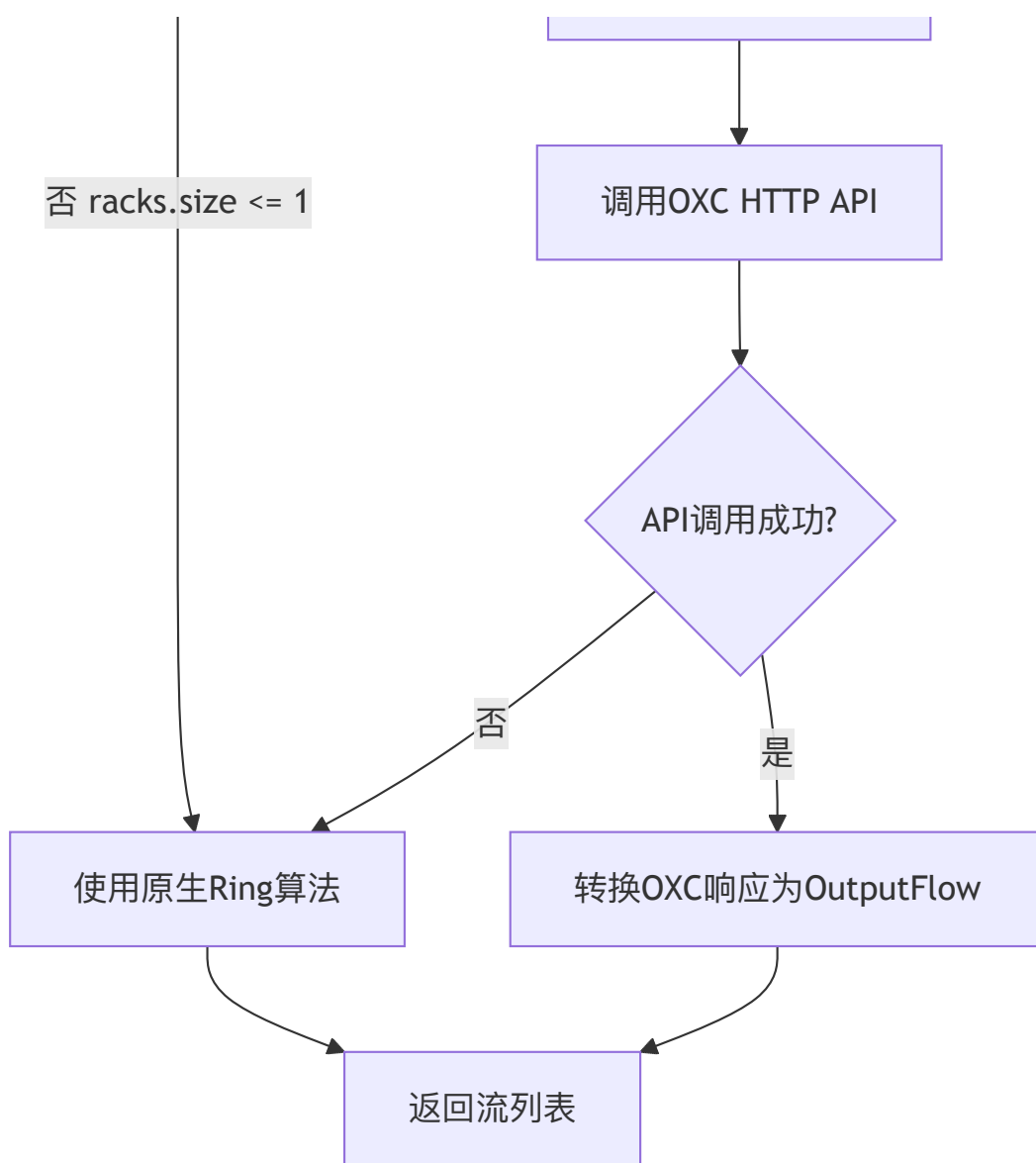


4.3 通信域构建流程图

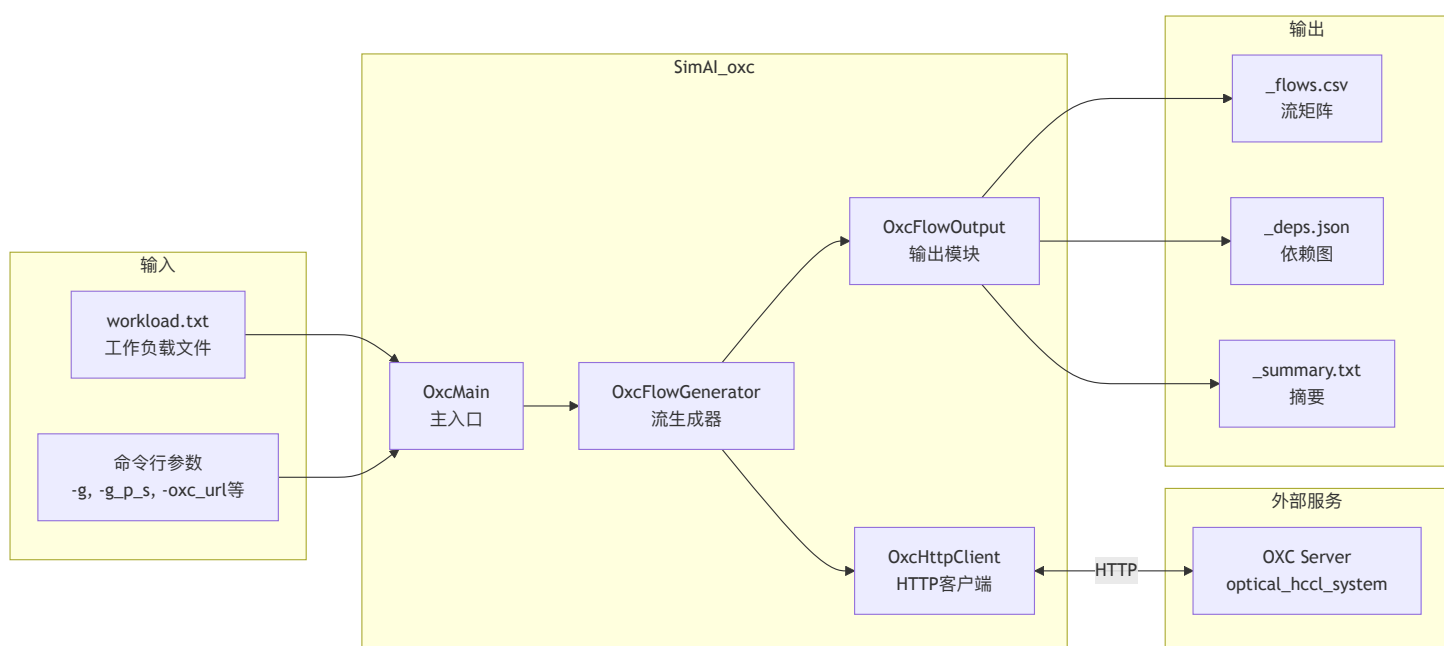


4.4 OXC vs 原生算法选择流程图





5. 数据流图



6. 文件结构

```
astrasim-alibabacld/
├── build/
│   ├── simai_oxc/
│   │   ├── build.sh          # OXC编译脚本
│   │   └── CMakeLists.txt    # OXC CMake配置
│   └── astrasim/
│       ├── system/
│       │   ├── OxcTypes.h    # OXC数据类型定义
│       │   └── network_frontend/
│       │       └── oxc/
│       │           ├── CMakeLists.txt    # OXC前端CMake配置
│       │           ├── OxcMain.cc       # 主入口
│       │           ├── OxcFlowGenerator.h/cc  # 流生成器
│       │           ├── OxcHttpClient.h/cc  # HTTP客户端
│       │           └── OxcFlowOutput.h/cc  # 输出模块
│       └── build.sh          # 主编译脚本（已修改）
└── scripts/
    └── build.sh              # 顶层编译脚本（已修改）
```

7. API 规范

7.1 OXC AllReduce API

端点: `POST /api/oxc/allreduce`

请求体:

```
{
  "ranktable": {
    "version": "2.0",
    "status": "completed",
    "rank_count": 16,
    "rank_list": [
      {
        "rank_id": 0,
        "device_id": 0,
        "local_id": 0,
        "level_list": [
          {
            "net_layer": 0,
            "net_instance_id": "superpod1_0",
            "net_type": "TOPO_FILE_DESC",
            "net_attr": "",
            "rank_addr_list": [
              {
                "addr_type": "EID",
                "addr": "00000000000000002000100000df001001",
                "ports": ["0/0"],
                "plane_id": "plane0"
              }
            ]
          }
        ]
      }
    ]
  },
  "dpCommDomain": [[0, 2, 4, 6, 8, 10, 12, 14]],
  "commDomainVolume": 16777216,
  "rankIdRackIdMap": {
    "0": "rack_0",
    "2": "rack_0",
  }
}
```

```
      "4": "rack_0",
      "6": "rack_0",
      "8": "rack_1",
      "10": "rack_1",
      "12": "rack_1",
      "14": "rack_1"
    },
    "algName": "ALGO_OXC_RING"
  }
}
```

响应体:

```
[
  [0, 8, 0, 2097152],
  [8, 0, 0, 2097152],
  [2, 10, 0, 2097152],
  [10, 2, 0, 2097152]
]
```

响应格式: [[src_rank, dst_rank, step, datasize], ...]

7.2 支持的算法

算法名称	描述
ALGO_OXC_RING	OXC优化的Ring算法
ALGO_OXC_HD	OXC优化的Halving-Doubling算法
ALGO_OXC_NB	OXC优化的非阻塞算法

8. 输出格式

8.1 流矩阵 CSV (_flows.csv)

```
op_id,layer,phase,comm_type,group,flow_id,src,dst,size,step,depends_on
0,embedding_layer,FORWARD,ALL_REDUCE,DP,0,0,8,2097152,0,"[]"
0,embedding_layer,FORWARD,ALL_REDUCE,DP,1,8,0,2097152,0,"[]"
0,embedding_layer,FORWARD,ALL_REDUCE,DP,2,2,10,2097152,0,"[]"
0,embedding_layer,FORWARD,ALL_REDUCE,DP,3,10,2,2097152,0,"[]"
0,embedding_layer,FORWARD,ALL_REDUCE,DP,4,0,8,2097152,1,"[0,1]"
```

8.2 依赖图 JSON (_deps.json)

```
{
  "operations": [
    {
      "op_id": 0,
      "layer": "embedding_layer",
      "layer_index": 0,
      "phase": "FORWARD",
      "type": "ALL_REDUCE",
      "group": "DP",
      "data_size": 16777216,
      "flow_count": 32,
      "depends_on": []
    },
    {
      "op_id": 1,
      "layer": "attention_column",
      "layer_index": 1,
      "phase": "FORWARD",
```

```
    "type": "ALL_GATHER",
    "group": "TP",
    "data_size": 8388608,
    "flow_count": 24,
    "depends_on": [0]
  }
],
"dependencies": {
  "1": [0],
  "2": [1]
}
}
```

8.3 摘要 TXT (_summary.txt)

SimAI-OXC Flow Generation Summary
=====

Workload Configuration:
Parallelism Policy: HYBRID_DATA_MODEL
Total GPUs: 16
GPUs per Server: 8
TP Size: 2
EP Size: 1
PP Size: 1
...

OXC Configuration:
Server URL: http://localhost:8080
Algorithm: ALGO_OXC_RING

Operations Processed:
Total Operations: 24
- ALL_REDUCE: 8 (OXC)
- ALL_GATHER: 8 (Native)
- REDUCE_SCATTER: 8 (Native)

Flow Statistics:
Total Flows Generated: 512
Total Dependencies: 384

9. 编译与运行

9.1 编译

```
# 编译 SimAI-OXC
./scripts/build.sh -c oxc

# 清理
./scripts/build.sh -l oxc
```

9.2 运行

```
./bin/SimAI_oxc \  
-w ./example/workload_analytical.txt \  
-g 16 \  
-g_p_s 8 \  
-oxc_url http://localhost:8080 \  
-oxc_algo ALGO_OXC_RING \  
-o ./results/oxc_output
```

9.3 命令行参数

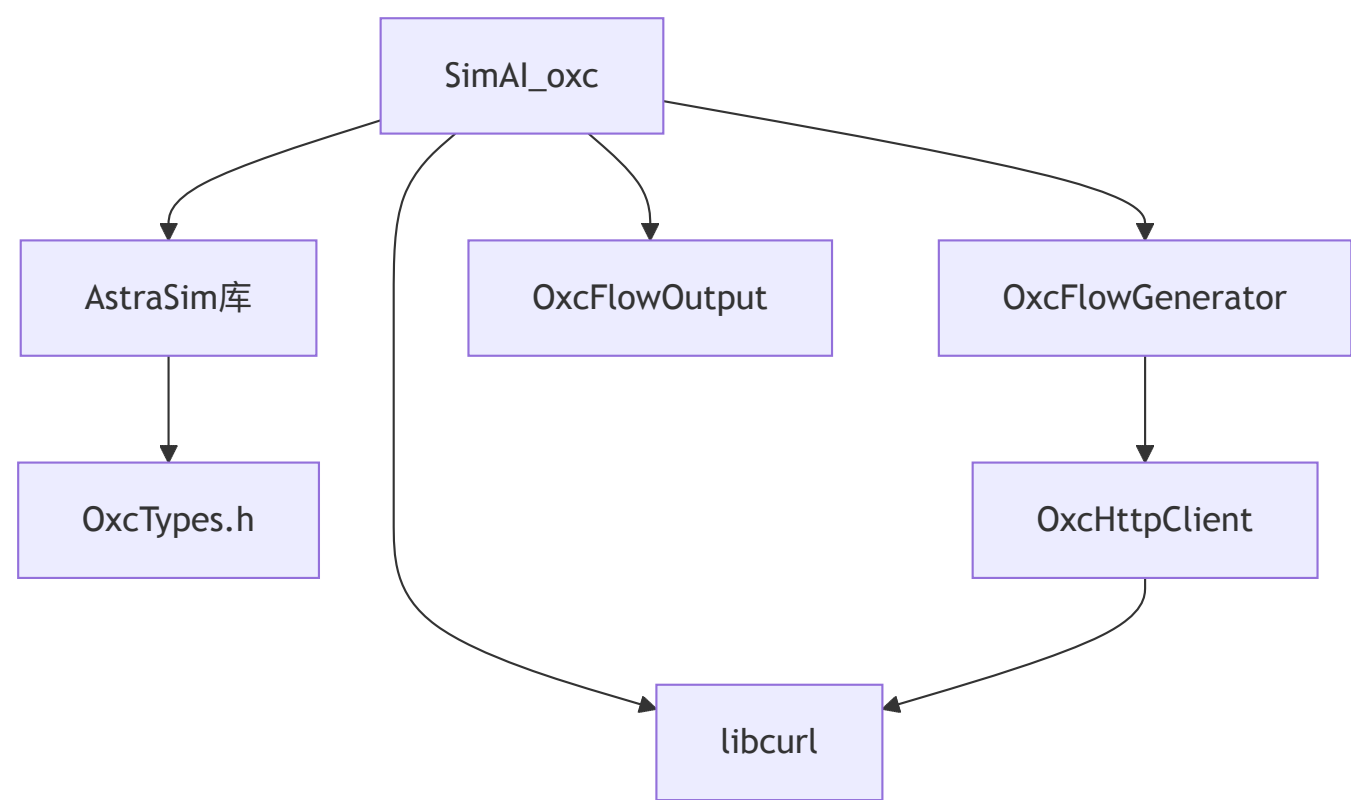
参数	说明	默认值
<code>-w, --workload</code>	工作负载文件路径	(必需)
<code>-g, --gpus</code>	GPU总数	16
<code>-g_p_s, --gpus-per-server</code>	每服务器GPU数	8
<code>-oxc_url</code>	OXC服务器URL	http://localhost:8080
<code>-oxc_algo</code>	OXC算法名称	ALGO_OXC_RING
<code>-o, --output</code>	输出文件前缀	./results/oxc_output

10. 依赖关系

10.1 外部依赖

- `libcurl`: HTTP客户端库
- `CMake >= 3.15`: 构建系统
- `C++17`: 编译器标准

10.2 内部依赖



11. 错误处理

错误场景	处理方式
OXC API调用失败	回退到原生Ring算法

工作负载文件不存在	报错退出
无效的命令行参数	打印使用说明并退出
HTTP连接超时	记录错误，回退到原生算法
JSON解析失败	记录错误，回退到原生算法

12. 扩展性

12.1 添加新的集合通信类型

1. 在 `OxcTypes.h` 中添加新的 `CommType` 枚举值
2. 在 `OxcFlowGenerator::isOxcSupported()` 中添加支持判断
3. 实现对应的 `generateXxxViaOxc()` 方法
4. 在 `generateViaNative()` 中添加回退实现

12.2 添加新的OXC算法

只需在调用时传入新的 `algName` 参数，无需修改代码。

12.3 添加新的输出格式

在 `OxcFlowOutput` 类中添加新的 `writeXxx()` 方法。

13. 关键设计决策

13.1 通信域全局缓存

通信域在 `OxcFlowGenerator` 构造函数中一次性计算并缓存到 `comm_domains_` 成员变量中，而不是每次调用时动态计算。这与 `MockNcclGroup` 的设计保持一致。

原因：

1. 通信域在整个仿真过程中是固定的（由 TP/DP/EP/PP 参数决定）
2. 避免重复计算，提高性能
3. 保证所有操作使用一致的通信域划分

实现：

```
// 构造函数中初始化
OxcFlowGenerator::OxcFlowGenerator(...) {
    initGlobalRankTable();
    initGlobalRankIdRackIdMap();
    initCommDomains(); // 一次性计算所有通信域
}

void OxcFlowGenerator::initCommDomains() {
    comm_domains_[GroupType::TP] = computeCommDomains(GroupType::TP, num_gpus_);
    comm_domains_[GroupType::DP] = computeCommDomains(GroupType::DP, num_gpus_);
    comm_domains_[GroupType::EP] = computeCommDomains(GroupType::EP, num_gpus_);
    comm_domains_[GroupType::DP_EP] = computeCommDomains(GroupType::DP_EP, num_gpus_);
    comm_domains_set_ = true;
}

// 使用时直接从缓存获取
std::vector<std::vector<int>>> OxcFlowGenerator::buildCommDomains(GroupType type, int total_gpus) {
```

```

if (comm_domains_set_) {
    return comm_domains_[type]; // 从缓存返回
}
return computeCommDomains(type, total_gpus); // 兼容旧代码
}

```

13.2 OXC 仅用于跨框通信

OXC 算法专为跨框（跨 rack）通信优化。对于框内通信，使用原生 Ring 算法更高效。

判断逻辑：

```

std::vector<OutputFlow> OxcFlowGenerator::generateAllReduceViaOxc(
    const OperationContext& ctx,
    const std::vector<int>& comm_group_ranks) {

    // 收集通信组中所有 rank 所在的 rack
    std::set<int> racks;
    for (int rank : comm_group_ranks) {
        int rack_id = rank / gpus_per_server_;
        racks.insert(rack_id);
    }

    // 如果所有 rank 都在同一个 rack, 使用原生算法
    if (racks.size() <= 1) {
        return generateViaNative(ctx, comm_group_ranks);
    }

    // 跨 rack 通信, 调用 OXC API
    // ...
}

```

示例（16 GPU, 8 GPU/server）：

- TP 组 `[0,1,2,3,4,5,6,7]`：全在 rack_0 → 使用原生算法
- DP 组 `[0,8]`：跨 rack_0 和 rack_1 → 使用 OXC API

13.3 全局 RankTable

RankTable 在构造函数中初始化一次，包含所有 GPU 的网络拓扑信息。每次 API 调用时复用全局 RankTable，只更新 `dpCommDomain` 字段。

优点：

1. 避免每次 API 调用时重复构建 RankTable
2. 保证所有请求使用一致的拓扑信息
3. 减少内存分配和 JSON 序列化开销

13.4 训练阶段与通信组映射

根据 `parseGroupType()` 函数的逻辑：

- **FORWARD_PASS** (前向传播) → TP 组
- **INPUT_GRADIENT** (输入梯度) → TP 组
- **WEIGHT_GRADIENT** (权重梯度) → **DP 组**

这意味着：

- TP 通信通常在框内（同一 server 的 GPU）
- DP 通信通常跨框（不同 server 的 GPU）
- 因此 DP AllReduce 是 OXC 优化的主要目标

13.5 错误处理与回退

当 OXC API 调用失败时，自动回退到原生 Ring 算法，确保流生成不会中断：

```
auto entries = http_client_.callAllReduceApi(request);

if (entries.empty()) {
    std::cerr << "[OXC] Warning: Empty response, falling back to native" << std::endl;
    return generateViaNative(ctx, comm_group_ranks);
}
```

14. 版本历史

版本	日期	变更
1.0	2024-01	初始版本，支持 AllReduce via OXC API
1.1	2024-01	添加通信域全局缓存，仿照 MockNcclGroup 实现
1.2	2024-01	添加跨框通信判断，OXC 仅用于跨 rack 通信